| # | Rol | Decisión | Estado | Lo que escribe el equipo |
|---|---|---|---|---|
| D1 | Hipotecas | El subsidio semanal lo calcula `Mortgage` | Tomada | Principio GRASP y qué datos de `Mortgage` lo justifican<br>**Respuesta:**<br>**Principio GRASP:** Experto en Información<br>La clase Mortgage (Hipoteca) es la que posee los atributos y datos requeridos para realizar el cálculo (monto de la hipoteca, tasa, plazos e información del titular). Por lo que la responsabilidad de calcular el subsidio semanal debe caer en ella para mantener la encapsulación y evitar exponer sus datos innecesariamente a clases externas. |
| D2 | Hipotecas | ¿El subsidio y el aporte de la pareja se guardan como atributos al construir la hipoteca, o se calculan cada vez que se piden? | Abierta | Opción elegida y por qué<br>**La opción elegida:** calcular cada vez que se piden (mediante métodos calculadores/getters)<br><br>Por qué: Evita la redundancia de datos y previene inconsistencias de estado en caso de que las variables de cálculo o políticas cambien. Además, favorece un menor acoplamiento. |
| D3 | Inversiones | `Investment` está declarada `final` | Tomada, revisable | ¿La mantienen así? Una ventaja y un riesgo<br>**Respuesta:**<br>Se mantiene final (o se remueve según la necesidad de extensión).<br><br>**Ventaja:** Garantiza la inmutabilidad y seguridad del diseño, previniendo alteraciones no deseadas mediante herencia (*Design for extension or else prohibit it*).<br><br>**Riesgo:** Restringe el polimorfismo. Si a futuro se requiere crear tipos específicos de inversiones mediante subclases (ej. RealEstateInvestment), la palabra clave final impedirá la extensión directa y obligará a refactorizar hacia interfaces o composición. |
| D4 | Registro | ¿Quién detecta un número de hipoteca repetido: | Abierta | Opción elegida, principio y qué pasaría si mañana cambia el repositorio<br>**Respuesta:** Experto en Información y Bajo Acoplamiento. |

| # | Rol | Decisión | Estado | Lo que escribe el equipo |
|---|---|---|---|---|
| | | `RegisterMortgage` o el repositorio? | | **Justificación:** El repositorio es la abstracción que gestiona la colección de hipotecas y conoce el conjunto de datos almacenados. RegisterMortgage (el caso de uso) solo consulta la existencia al repositorio antes de proceder a la creación.<br><br>**Si mañana cambia el repositorio:** Gracias al principio de **Variaciones Protegidas (Protected Variations)**, como el caso de uso interactúa a través de una interfaz abstracta, cambiar la tecnología de almacenamiento subyacente no impacta ni requiere modificar la lógica de negocio en RegisterMortgage. |
| D5 | Integrador | `EstimateFundsForWeek` recibe `MortgageRepository` (una interfaz) y no `InMemoryMortgageRepository` | Tomada | Principio GRASP y qué gana el caso de uso con esa decisión<br>**Respuesta:** Variaciones Protegidas y Bajo Acoplamiento<br><br>**Qué gana el caso de uso:** Independencia de la tecnología de persistencia. Permite cambiar la implementación concreta e inyectar mocks/stubs para pruebas unitarias sin modificar el código del caso de uso. |
| D6 | Integrador | Las sumas sobre todas las hipotecas e inversiones están en métodos privados de `EstimateFundsForWeek`. ¿Se quedan ahí o pasan a un experto nuevo del dominio? | Tomada, revisable | Opción elegida y por qué<br>**Respuesta:** Pasan a un nuevo experto del dominio (o servicio de dominio / fabricación pura).<br>**Por qué:**<br>**Alta Cohesión (High Cohesion):** La responsabilidad principal de EstimateFundsForWeek es coordinar el flujo del caso de uso. Realizar cálculos numéricos e iteraciones detalladas sobre colecciones disminuye su cohesión.<br><br>**Reutilización:** Al mover los cálculos a un experto o servicio del dominio, la lógica queda centralizada y disponible para otros casos de uso o reportes sin duplicar código. |